



The Metaflow Architecture

The Metaflow architecture exploits inherent instruction-level parallelism in conventional sequential programs by hardware means, without relying on optimizing compilers. It executes instructions out of order and speculatively via DRIS, an integrated instruction-shelving structure. The first implementation of the architecture is Lightning, a superscalar Sparc processor.

Val Popescu

Merle Schultz

John Spracklen

Gary Gibson

Bruce Lightner

David Isaman

*Metaflow
Technologies, Inc.*

The Metaflow architecture is a unified approach to maximizing the performance of superscalar microprocessors by using the techniques of out-of-order execution and multilevel speculative execution. A superscalar microprocessor attempts to execute a conventional sequential instruction set at an improved level of performance by executing, on average, more than one instruction per clock cycle.

As is common with most other performance-enhancing techniques, superscalar architectures rely on two primary elements: having multiple execution units and keeping those units operating concurrently much of the time. Distinguishing a superscalar architecture from others, however, is its maintenance of binary compatibility with industry-standard processors. This compatibility contrasts with those machines—parallel processors, vector processors, very-long-instruction-word machines—that require at least recompilation, if not complete restructuring, of programs.

Another approach that delivers equivalent performance and binary compatibility is to run a scalar (one instruction at a time) implementation at a much higher clock rate; this requires pipelining the processor more heavily (superpipelining).¹ Both superscalar and superpipelined machines share an inherent limitation, however. Whenever the processor encounters an instruction that depends on the output of some unfinished instruc-

tion, the processor stalls. A stall introduces clock periods during which no instructions start executing, resulting in less than optimum performance.

Avoiding a stall requires that 1) the program has sufficient instruction-level parallelism that instructions independent of the unfinished instructions exist and 2) those independent instructions appear before any dependent instructions. Compiler techniques that exploit instruction-level parallelism to reduce stalling are well-known.² Upon encountering an instruction that depends on an instruction likely still to be in execution, the compiler can check for later independent instructions and move them ahead of the dependent instruction.

Relying on compilers to obtain maximum performance has several drawbacks, however. Such an approach requires access to the source code, it increases compilation time, and it may require subsequent recompilations as each new generation of processor appears.

An alternative is to build processors in which the hardware will search past a dependent instruction for later independent instructions and execute them earlier. This feature reduces the performance loss due to stalls, but to be fully effective, such an out-of-order execution machine must solve several other problems, including artificial dependencies caused by register-set limitations, stalls caused by conditional branches, and imprecise interrupts.

This article introduces the Metaflow architecture. (Certain key aspects of the Metaflow architecture described here are the subject of a pending patent application.) This new processor implementation architecture is based on a unified structure, the DRIS (deferred-scheduling, register-renaming instruction shelf), that manages out-of-order execution and most of the attendant problems. Coupling the DRIS with a unique speculative-execution mechanism that avoids conditional branch stalls results in performance limited only by inherent instruction-level parallelism and available execution resources. Although presented here in the context of superscalar machines, the technique is equally applicable to a superpipelined implementation.

Out-of-order execution

Through a series of simple examples, we can explore the problems that arise in out-of-order execution and the techniques traditionally used to solve them. The examples assume a hypothetical three-address, reduced-instruction-set computer (RISC) with eight registers. The basic pipeline assumed appears in Figure 1. The primary hardware elements include an instruction cache, a register file, and one or more execution units, all separated by pipeline registers.

The cache is crucial. It avoids stalling for lack of instructions, especially in a superscalar machine that consumes instructions at a prodigious rate. The register file provides storage for the eight registers. The single execution unit shown represents a set of distinct execution units, including an integer ALU, a path to memory through an address ALU (which may be the same as the integer ALU), and possibly one or more floating-point units.

Each instruction in this simple machine goes through four phases: fetch, issue, execute, and update. In the fetch phase the processor fetches an instruction from the instruction cache (discussion of this logic appears later). The issue logic partially decodes the instruction, to determine the type of execution unit needed, and reads the instruction's operands from the register file.

The execute phase, unlike the other three, can take more than one clock cycle to complete. Simple integer arithmetic and logic operations generally finish in one clock cycle, but loading data from memory takes longer; the examples here assume a minimum of five clock cycles. Similarly, floating-point operations typically take at least three or four cycles.

The final phase, update, consists of writing the result of the instruction execution into its destination register's location in the register file. The bypass path permits using the result of execution on one clock cycle as an operand on the next cycle.

Data dependency and stalling. The bypass path guarantees that the issue logic may send one new instruction into execution every clock cycle if each instruction's execution requires only one cycle. To control this bypass path, the is-

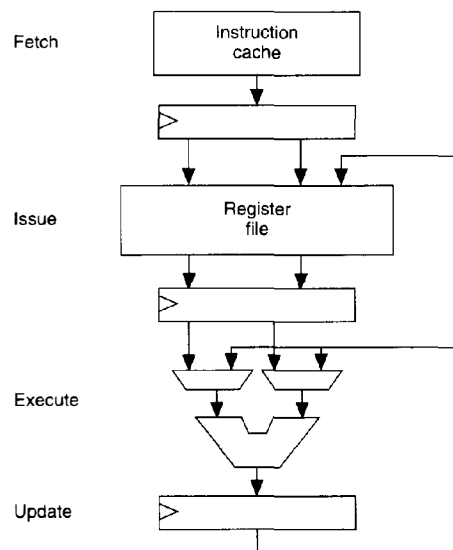


Figure 1. Basic pipeline.

sue logic must check for a data dependency between the instruction it is issuing and the one it issued on the previous cycle. A data dependency exists between two instructions if the execution of one depends on the results of executing the other.

An instruction that takes more than one clock cycle (such as a load) has the potential to stall later instructions that depend on it, as illustrated in Table 1. (The machine code syntax shown in Tables 1 through 3 specifies an opcode followed by two source registers and then a destination register. For example:

Table 1. Stalling for data dependency.	
Opcode	Operands
ADD	r1,r2,r3
LOAD	[r3],r6
AND	r5,3,r7
ADD	r6,r7,r1
SUB	r3,r4,r5

Table 2. Out-of-order execution.

Opcode	Operands	Phase			
		Fetch	Issue	Execute	Update
ADD	r1,r2,r3	0	1	2	3
LOAD	[r3],r6	1	2	3	9
AND	r5,3,r7	2	3	4	5
ADD	r6,r7,r1	3	4	9	10
SRL	r0,8,r2	4	5	6	7
OR	r4,r2,r2	5	6	7	8
SUB	r3,r4,r5	6	7	8	9

ADD r1,r2,r3 specifies that the sum of registers r1 and r2 transfers to register r3. LOAD [r3],r6 requests a load of register r6 with the contents of the memory location specified by r3.) In that program, the second ADD instruction depends on the LOAD instruction for one of its operands, r6. Because the LOAD instruction takes five clock cycles, and the issue logic encounters the dependent ADD instruction after two clocks, the issue logic must stall the machine for three clock cycles.

The issue logic must stall any instruction having an operand that is locked (that is, the preceding instruction that is to produce that operand has not yet done so).

Instruction shelving. Table 2 demonstrates how out-of-order execution can overcome the performance loss caused by stalling. We have augmented the program listed in Table 1 with two instructions at the end that are independent of all preceding instructions. A timing chart shows the clock cycle at which each instruction begins each phase.

The principle of out-of-order execution is that the issue logic shelves any stalled instruction and continues to issue subsequent instructions, returning to the shelved instruction only after all its operands are available. The effect of this in Table 2 is that at clock cycle 4 the issue logic shelves the dependent ADD instruction, and then moves on to issue the following three independent instructions, which enter execution at clocks 6, 7, and 8. The LOAD finishes execution at clock 8, so the dependent ADD can be launched into execution on clock 9.

This example reduces the three-clock stall to one, clock 5, at which no instruction starts execution. To completely mask that loss, two instructions must enter the execute phase at some later clock cycle. The first time that two candidates are available is clock 9. The first candidate is the shelved dependent ADD, while the second candidate is the next instruction in sequence after the SUB instruction.

Launching both of those instructions into execution on clock 9 requires that:

- 1) the next sequential instruction does not itself need to be shelved,

- 2) sufficient execution resources exist, and
- 3) control logic capable of matching the instructions to the resources exists.

Requirement 1 depends on the program; later we discuss techniques for increasing the number of candidate instructions to maximize the chances of finding an independent one. Requirement 2 depends on both the program and the resources available. If the two candidate instructions require different types of execution resources, the requirement is automatically satisfied. If they need the same type of execution unit, the processor must have more than one of those units. One advantage of out-of-order execution is that it lets duplicate execution units run in parallel.

Launching shelved instructions into execution concurrently with other instructions requires that shelved instructions undergo a new phase of processing between the issue and execute phases. This scheduling phase occurs when a shelved instruction is ready to execute—all its operands are no longer locked. Scheduling consists of reading the operands, determining the resources needed by the ready instruction, and resolving any resource conflicts between it and other ready instructions.

The principle of shelving instructions whose operands are locked at issue time is credited to Tomasulo,³ who implemented a small instruction shelf called a reservation station in the floating-point unit of the IBM 360 Model 91.

Register renaming. Executing instructions out of order can easily introduce new, artificial dependencies between instructions. Such a dependency would occur if, for example, the register used in Table 2 to pass the result of the SRL instruction to the following instruction were changed from 2 to 7. If this modified program were executed with the timing shown in Table 2, the result of the SRL would update register 7 of the register file at clock cycle 7. This update would occur one cycle before the operands of the shelved ADD, including r7, are read out of the register file. Avoiding this conflict requires detecting such antidependencies and shelving the later instruction; in this case, SRL must be shelved for at least one clock period.

Providing additional register storage and renaming registers will avoid stalling caused by antidependencies. To avoid stalling SRL in this example, 1) rename the destination register of SRL to r7' (one of the additional registers), and 2) rename to r7' the operand of all succeeding instructions that use the output of SRL (all instructions through the next one to write r7).

Making effective use of additional register storage through register renaming requires still more sophisticated control structures.⁴ The issue logic can easily detect, when it sees an instruction that writes r7, that the current name for r7 is locked. It can then lock and start to use the alternate name, if the alternate name is not already locked. Two more difficult

Table 3. An in-order superscalar schedule.						
Clock cycle	Opcode	Operands	Phase			
			Fetch	Issue	Execute	Update
0	ADD	r1,r2,r3	0	1	2	3
1	LOAD	[r3],r6	0	1	3	9
2	AND	r5,3,r7	1	2	3	4
3	ADD	r6,r7,r1	1	2	9	10
4	SRL	r0,8,r7	2	3	4	5
5	OR	r4,r7,r2	2	3	5	6
6	SUB	r3,r4,r5	3	4	5	6
7	ADD	r1,10,r0	3	4	10	11
8	LOAD	[r5],r6	4	5	6	12
9	SUB	r1,r6,r2	4	5	12	13
10	AND	r7,15,r3	5	6	7	8

problems are unlocking a register name and avoiding stalls when both names for a register are locked.

A register name is unlocked when all instructions that use that name as an operand have actually read their operands. Out-of-order execution complicates the determination that a given use is the last use: The order in which the two names for r7 are unlocked can be the opposite of the order in which they were locked.

Avoiding stalls when a third name for r7 is needed requires a more general allocation scheme for register names, including maintaining a mapping from program register names to arbitrary physical registers (instead of the single bit needed to distinguish r7' from r7, for example).

Register renaming is another innovation due to Tomasulo. His algorithm renames a register to the unique identifier of the reservation station holding the (most recently issued) instruction to write that register. The actual additional register storage occurs in the reservation stations that use the contents of the renamed registers.

Decoupled scheduling. The preceding examples assumed a simple scalar machine, issuing one instruction per clock cycle. The timing chart in Table 3 lists the performance possible from a superscalar implementation that attempts to

Table 4. An ideal schedule.	
Clock cycle	Instruction nos.
2	0, 2
3	1, 4
4	5, 6
5	8, 10

issue and schedule two consecutive instructions per clock cycle.

This particular instruction stream includes data dependencies between consecutive instructions. Combining these with the issue bandwidth lost to shelving instructions results in only six instructions being sent to execution during the first four clock cycles (2 through 5). An ideal scheduler, constrained only by true data dependencies, could have scheduled eight instructions during those first four cycles, as listed in Table 4.

The tight coupling between the issuing and scheduling of unshelved instructions causes the loss of performance evidenced in this example. Since the issue logic is restricted to a window of

two consecutive instructions, it does not "see" that, for example, two of the first three instructions are independent and so could be scheduled concurrently.

The solution to this limitation is to decouple scheduling from issuing, shelving every instruction upon issue. Those instructions that could have been sent immediately to execution are instead shelved, premarked as ready.

The separate scheduler can be designed with a wider window. In this example, a window of three provides ideal scheduling for the first four clock cycles, but for the next three, no instructions will be ready for scheduling. Sustaining a high use of execution units requires a much wider scheduling window.

Dataflow scheduling. The preceding examples have all assumed that the issue logic could determine—for each instruction that it needed to shelve—when the instruction would be ready to execute. Thus when the superscalar logic in Table 3 issues the dependent ADD instruction in clock cycle 2, it needs information that the result of the LOAD issued at clock cycle 1 will be available on clock cycle 9. In a real machine with a data cache, however, such certain knowledge is not immediately available.

The execution unit (the memory subsystem) itself determines the execution time of the LOAD instruction, based on the operands of the instruction. Since the execution unit establishes the data-arrival time, it must inform the scheduler of that arrival. The simplest conceptual picture is that the execution unit carries the destination register name (appropriately renamed) with the instruction throughout the variable-length processing cycle. When that cycle completes, the destination register name accompanies the result data.

When an execution unit presents a completed instruction's destination register name, the update logic must search

continued on p. 63

Metaflow architecture

continued from p. 13

through all shelved instructions, unlocking any operand locked on that register name. When any instruction's last remaining locked operand is unlocked by this action, that instruction can execute.

Scheduling an instruction for execution only when all its operand values have been produced by execution units operating on other instructions is the essence of dataflow scheduling.

Precise interrupts. Table 3 also illustrates one final problem in the design of an out-of-order machine. If the LOAD instruction (instruction 1) fails—for example, encounters an uncorrectable memory error—it must cause an interrupt. The need to interrupt will not be known until the same time that the data would have arrived, clock 9. But, as the update column shows, the results of five later instructions will already have been written to the register file. As a consequence, any attempt to restart execution at the interrupted instruction will fail because the four registers written by those five later instructions contain the wrong values.

The problems caused by writing an instruction's result to the register file at its update phase suggest that results also should be shelved for a period of time before writing them to the register file. This step adds a sixth phase of processing each instruction, the retire phase. At this phase the instruction's result moves from the result shelf into the register file. Postponing retirement until it is known that no older instruction can cause an interrupt guarantees a coherent register file; the same rule ensures a coherent memory content as well.

Shelving an instruction's result introduces new problems. When the update phase unlocks an operand locked on a result, it must place with the operand on the instruction shelf an indication of where to find that result: at some location on the result shelf. Then when that result is finally retired to the register file, the retire logic must find any dependent instruction still waiting to be scheduled and change its operand location from the result shelf to the register file.

Smith and Pleszkun⁵ describe the principle of shelving results in a "reorder buffer." They address only the problem of precise interrupts in the basic pipeline of Figure 1, that is, a machine that stalls the issuing of instructions when an operand is locked.

The DRIS

A recurrent theme throughout the preceding discussion of problems inherent in out-of-order execution is that of shelving information for later processing. The issue logic places instructions that depend on other unfinished instructions for their operands on an instruction shelf, to be scheduled later when all operand data is available. Scheduling efficiency in-

creases if all instructions are shelved at issue time, so that the scheduler can choose from a large number of candidate instructions each clock cycle. The results of an instruction must be shelved on a result shelf until the results of all earlier instructions have been retired to the register file.

The DRIS (deferred-scheduling, register-renaming instruction shelf), depicted in Figure 2, is a single integrated shelving structure that provides for straightforward solutions to all of the control problems we've described. Instructions move into the DRIS at issue time in the order in which they were issued. They stay there until they are removed, in that same order, at retire time.

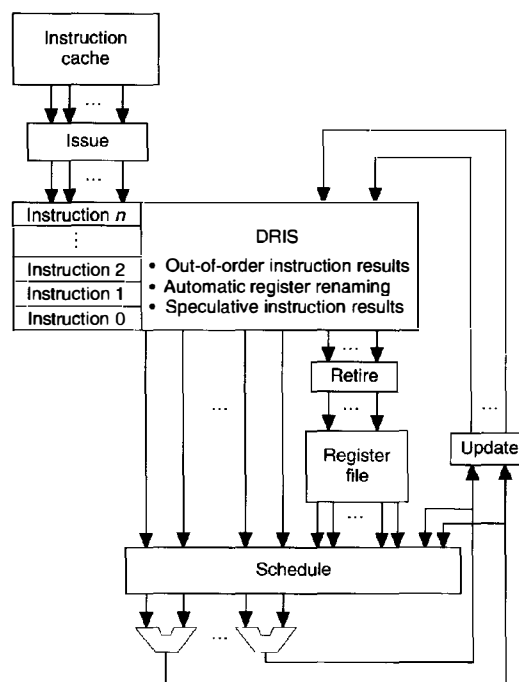


Figure 2. The DRIS.

Entering an instruction into the DRIS automatically and uniquely renames its destination register. The issue logic determines the true data dependencies on earlier uncompleted instructions by searching the DRIS. When the instruction's result becomes available at update time, the value is shelved adjacent to the instruction in the DRIS. The unique destination register name for that result is used at update time to

Source operand 1			Source operand 2			Destination		
Locked	Reg num	ID	Locked	Reg num	ID	atest	Reg num	Content

Dispatched	Functional unit	Executed	Program counter
Yes/no	Class number	Yes/no	Content

Figure 3. An entry in the DRIS.

search the DRIS for operands locked on that result, and searching for instructions with all operands unlocked accomplishes the deferred scheduling.

Figure 3 depicts the format of an entry in the DRIS. The following sections explain the meaning and function of each field in conjunction with the descriptions of the five control processes that interact with the DRIS. These processes correspond to the five stages of processing an instruction after it is fetched: Issue, Schedule, Execute, Update, and Retire.

Issue. The Issue process is superscalar, issuing multiple instructions per clock cycle. It writes each issued instruction into the DRIS (unless the DRIS is full, in which case stalling is necessary). Issue also writes the register numbers for the two source operands and the destination into the respective Reg Num fields. It also writes a code for the class of functional unit required to execute the instruction into the functional unit field and resets the dispatched and executed bits.

The Issue process performs four additional functions—ID allocation, register renaming, dependency checking, and program counter maintenance.

ID allocation. Each issued instruction receives an identifier (ID) unique among all shelved instructions that consists of an index into the DRIS plus a “color” bit. The Issue process allocates IDs in strict numerical order, toggling the color bit when the index portion wraps around. The color bit simplifies the process of determining the relative age of two shelved instructions. Any process does this by comparing the index portion of their IDs and inverting the sense of the comparison if the IDs are of opposite colors. (Of any two instructions in the DRIS, the one issued first is the older, and the more recently issued one is the younger.)

Register renaming. The DRIS automatically renames registers by allocating storage (in the content field of the destination section) for the result of every yet-unretired (shelved) instruction. The ID allocated to the instruction identifies the alternate register storage by uniquely identifying the DRIS entry.

Whenever the Issue process issues an instruction that writes, for example, *r7*, it is guaranteed to have storage for a new instantiation of *r7* and a unique new name for the instantiation. The instantiation automatically allocated at issue time is auto-

matically deallocated as the instruction exits the DRIS at retire time.

Dependency checking. For each operand of each instruction issued, the Issue process searches for the operand's register number in the destination Reg Num fields of the DRIS. If no match is found, no instruction in the DRIS writes that register, so the operand's value is already in the register file. Otherwise the correct value will be found in the DRIS entry for the youngest instruction that does write the operand register; this instruction is termed the “locker” of the operand value.

In this latter case, the process sets the ID field in the appropriate operand section to the ID of the locker. The operand is locked unless the locker has already produced a value in that location, as indicated by its executed bit being set.

The latest bit in the destination section of the DRIS entry indicates that the youngest in the DRIS that writes its destination register. Thus the bit is set as the instruction is shelved in the DRIS at issue time and cleared when any subsequent instruction that writes the same register is issued.

If more than one instruction is being issued in one clock cycle, the process also compares the operands of each against the other instructions being issued concurrently.

Maintaining the program counter. Every instruction implicitly changes the program counter, if only to increment it by the length of the instruction. The Issue process must compute the effect of each instruction on the program counter before it can direct the fetching of the next instruction. This program counter result may be needed at a later time, however, if that next instruction causes an interrupt. Therefore the Issue process shelves the new program counter along with the instruction.

Schedule. The scheduler selects the instructions to be sent into execution next and fetches the operands of those instructions. It examines all instructions in the DRIS in parallel each clock cycle, looking for any whose operands are all unlocked. It sends the oldest among those into execution during the next cycle.

The scheduler is sensitive to the class of execution unit needed by each instruction. Thus, if the machine has two integer ALUs, the scheduler sends the two oldest ready integer instructions to those ALUs, possibly skipping over older

ready instructions for which no resources of the needed class are currently available.

The operand fetch step of the Schedule phase checks the DRIS entry of the scheduled instruction for the location of each operand, either in the register file or in the DRIS. A data path analogous to the bypass in Figure 1 allows the use of one execution result as an operand to another execution starting on the next clock cycle. The scheduler passes the operand values, ID, and opcode of each scheduled instruction to an execution unit. Finally, it sets the dispatched bit to prevent scheduling the instruction more than once.

Execute. The execution units, in keeping with the dataflow paradigm, effectively carry the ID of each instruction along with the operands, so that the ID arrives at the DRIS at the same time as the result data. For computational units, a simple shift register accomplishes this. The execution of memory access instructions occurs in two phases, as described later.

Update. The execution units present instruction IDs together with the results of those instructions to the Update process. Update writes each instruction's result back into the DRIS entry for that instruction and sets the executed bit. Update also uses the ID, which is the renamed destination register number, to search the DRIS for, and unlock, any operands that were locked on that ID; this may result in making one or more instructions newly ready to execute.

Retire. This process examines the oldest instructions shelved in the DRIS. At each clock cycle, it retires all instructions for which

- 1) the executed bit is set,
- 2) all older instructions have been retired or are being retired in this clock cycle,
- 3) the instruction did not result in an error condition that should cause an interrupt, and
- 4) a write port exists to the register file not used by an older instruction being retired during the same cycle.

Instructions that do not write the register file (compares, branches, stores) can be retired at the same time as those that do, as long as all older instructions are also being retired.

If the Retire process encounters an interrupting instruction, it waits until it has retired all older instructions before it signals the interrupt. Therefore, at interrupt time, the register file and memory content are coherent; that is, they reflect the result of executing all instructions before the interrupting instruction and none of the instructions after it. The Retire process supplies the program counter value for that instruction, which had been shelved with the next-older instruction by the Issue process. When the Retire process signals an interrupt, it also flushes all instructions from the DRIS, since they never should have been fetched.

Memory reference instructions. Executing these instructions differs substantially from register-to-register operations.

Loads must effectively go through the Issue, Schedule, Execute, and Update phases twice: first to compute the address, then again to fetch the memory data. Also, a significantly different form of data dependency governs the second cycle.

The first Issue phase for a load or store instruction discovers any instruction locking an operand of the instruction, as for other instruction types, simply by comparing register numbers from the instructions. In a load's second Issue phase, its operand resides in memory. That location is locked if any older unexecuted store instruction might store into that location. (A store instruction does not write to memory until retire time, so that memory will also be coherent.) Discovering whether that memory location is locked requires comparing its address against the addresses in all older shelved store instructions.

The actions during each phase of processing a load or store instruction are:

- **First issue.** Allocate an ID and determine the data dependencies of the operands used to compute the instruction's target address.
- **First schedule.** Schedule the memory reference instruction when all the operands needed for its target address calculation are unlocked.
- **First execute.** Compute the target address using an integer ALU.
- **First update.** Write the computed target address back into the DRIS adjacent to the memory reference instruction. Compare the computed address of a load instruction against the addresses of all older store instructions in the DRIS. Compare the computed address of a store instruction against that of all younger load addresses. In either case, lock any known load instruction address operand on any older store instruction having a known identical address operand.

The Retire phase, which writes its data operand to its target address in memory, immediately follows the first Update phase of a store instruction. The first Update phase of a load instruction is effectively its second Issue phase, in that a previously unavailable operand designator, the target address, is being entered into the DRIS. A second Schedule and Update phase follows this second Issue phase of a load:

- **Second schedule.** Schedule a load instruction the second time after its target address is known, all older store instructions' target addresses are known, and the load's target address is not locked.
- **Second execute.** Send the load instruction's one operand, the target memory address, to memory and receive the addressed data in return. Present the ID of the load instruction to the Update phase at the same time as the data (details of this coordination will vary with memory subsystem design).

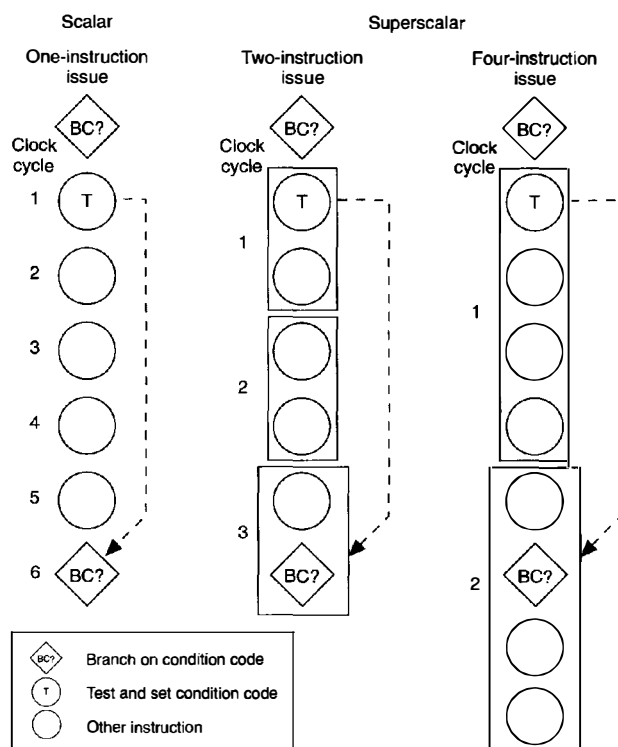


Figure 4. Stalls for conditional branches.

The second Update phase of a load instruction is identical to the single Update phase of a register-to-register instruction. The Retire phases are also identical.

The register update unit. Sohi⁶ describes the register update unit (RUU). One variant of this structure, dubbed RUU with bypass, achieves the objective of out-of-order execution with precise interrupts in a manner quite similar to that of the DRIS. The subtle differences between the two schemes, however, have two effects. The RUU is more costly to implement (at least in a microprocessor). More importantly, the RUU would have great difficulty in achieving the second major accomplishment of the Metaflow architecture, speculative execution.

The RUU with bypass differs from the DRIS in two areas, register renaming and operand storage and fetching. It names multiple instantiations of a register in a straightforward manner. The register number is concatenated with an incrementing index of the latest instance of that register. This process requires only two associative searches of the RUU at issue time,

compared to three for the DRIS, but those two searches are wider, hence slower. It also adds the cost of two counters per register, one for the number of outstanding instantiations and one for the latest instantiation. Finally, it compromises performance by introducing a new limited resource (instantiation numbers) that can result in stalling.

The second difference in the RUU is that it stores with each instruction not only the value of the instruction's result but also the value of both its operands. This added storage avoids the complication of determining at schedule time whether the operand remains on the shelf or has been retired. But storing the operands' values greatly increases the size of an RUU entry and also increases the amount of memory that must be written concurrently. At update time, a new result value may have to be written into any number of operand storage locations, in addition to the one destination storage location required for the DRIS. This requirement may have severe implications for power consumption and speed.

Speculative execution

A goal of a high-performance processor architecture is to avoid stalling, keeping a steady supply of candidate instructions available to be scheduled to multiple concurrent execution units. We've already introduced several techniques for avoiding stalls due to data dependencies: out-of-order execution, register renaming, multiple-instruction issue, and decoupled dataflow scheduling. We now address a technique for avoiding stalls due to control dependencies.

The most common control dependency is that of a conditional branch instruction on the result of some earlier computation. That result is conceptually a condition code, even in an instruction set without program-visible condition codes. Visible condition codes are typically a field in the processor status register. The DRIS in such a machine treats this field like an additional, implicitly addressed destination register. The values of condition codes produced by an instruction are shelved adjacent to it in the DRIS until the instruction is retired; retiring updates the coherent processor status register.

The Issue process must execute branches immediately, to direct fetching of subsequent instructions from the correct locations. Stalling arises if some older instruction locks the condition codes. Figure 4 demonstrates the potential performance impact of stalling conditional branches for machines that issue one, two, and four instructions per clock cycle.

Figure 4 assumes a best-case instruction set, in which the

instruction that sets the condition code can be separated from the conditional branch that depends on it. The figure shows only a five-instruction separation; since statistics show that an average basic block is fewer than eight instructions,⁷ this represents a realistic best case for many branch instructions.

The scalar machine has five clock cycles in which to compute the condition code before it is needed; a good chance exists that this is sufficient time to avoid stalling. But the superscalar machines may take only one or two cycles between issuing the condition code-setting instruction and needing its result. Stalling in this case results in a significant loss in performance.

Fortunately, simple techniques for branch prediction based on dynamic program behavior yield 85-95 percent prediction accuracy.⁸ Thus a branch instruction proceeds without stalling even when the condition codes are locked. Issuing and executing instructions following a conditional branch that is locked on condition codes is termed speculative execution.

We can view speculative execution as a special case of out-of-order execution. If a branch instruction is locked at Issue time, it is shelved for later execution, when the condition codes on which it is locked become available. The ID of the youngest instruction to write the branch's operand (the condition codes) is shelved with the branch instruction, along with one additional piece of information: the predicted branch direction.

At the Update phase of each instruction that sets condition codes, the instruction's ID searches for and unlocks all branch operands locked on it. A branch is scheduled if it is the oldest branch whose operand has become unlocked.

Executing a shelved branch determines which direction it should have taken. If that decision matches the direction the branch did take, the speculative execution was correct, so no further action is needed. If executing a speculative branch reveals that the prediction was incorrect, a branch repair mechanism is invoked to delete incorrect entries from the DRIS.

The process of branch repair undoes the effects of all instructions that were issued after an incorrectly predicted branch. Branch repair must be rapid to ensure that the performance gains of speculative execution are not cancelled. Hwu and Patt⁹ discuss branch repair in the context of an out-of-order execution mechanism in which all results are written to the register file immediately, rather than being shelved. They suggest a history buffer⁵ to repair main memory and a copy technique for registers.

The history buffer records, for each unretired instruction, the value contained in the instruction's destination just before the instruction overwrote it. Repairing requires the sequential restoration of values from the history buffer, which violates the requirement that repair be rapid. The copy technique for registers employs a specialized register file that maintains, in addition to the current working copy, one com-

plete checkpoint copy of all registers per predicted branch. This technique is a very expensive solution for a large register set.

Sohi points out that when the results of instructions are shelved before being retired to the register file, branch repair is a simple matter of deleting shelved entries for the instructions following an incorrectly predicted branch. This very fast operation requires no additional backup storage for instruction results. Sohi, however, offers no details of how the RUU could support branch repair, and it would appear that the chosen register-renaming scheme renders repair impractical. The values of the two counters per register would have to be restored, presumably using a copy technique, which again is expensive. Furthermore, all copies of the number-of-instances counter must be decremented each time an instance is retired.

The register-renaming technique used in the DRIS yields by far the simplest branch-repair mechanism. The ID of an incorrectly predicted branch identifies the entries that must be deleted. (Sohi suggests adding a field to the RUU to identify these entries.) That ID also becomes the next register instance number to use.

The Lightning module

Lightning, the first implementation of the Metaflow architecture, executes the Sparc RISC instruction set.¹⁰ Sparc provides register-register, register-immediate, and register-memory instructions. Most instructions contain three five-bit register addresses. For integer instructions, 24 of the 32 register addresses constitute a window into a much larger register file (128 registers in Lightning). The remaining eight registers are global to all windows. Floating-point instructions address a separate set of 32 floating-point registers, and condition codes are program-visible.

Lightning is a superscalar machine issuing up to four instructions per clock cycle. It is based on a structure called the DCAF (dataflow content-addressable FIFO) that implements the DRIS functions. Other elements of Lightning include two integer ALUs, one memory address ALU, an integrated floating-point unit, a Sparc reference memory management unit (MMU), and up to one megabyte of cache memory. Lightning consists of four ASICs (application-specific ICs) and an external cache (see Figure 5 on the next page) packaged as a module that interfaces to the Sun Microsystems proprietary Mbus. The four ASICs include the instruction issue unit, the dataflow integer unit, the floating-point unit, and the cache controller/MMU/bus interface (CMB). The external cache consists of from 128 Kbytes to 1 Mbyte of static RAM; this is the first-level cache for data and the second-level cache for instructions.

The Fetch process resides in the instruction issue unit. The Issue process is replicated in this unit, the dataflow integer unit, and the floating-point unit. The Schedule, Execute, and

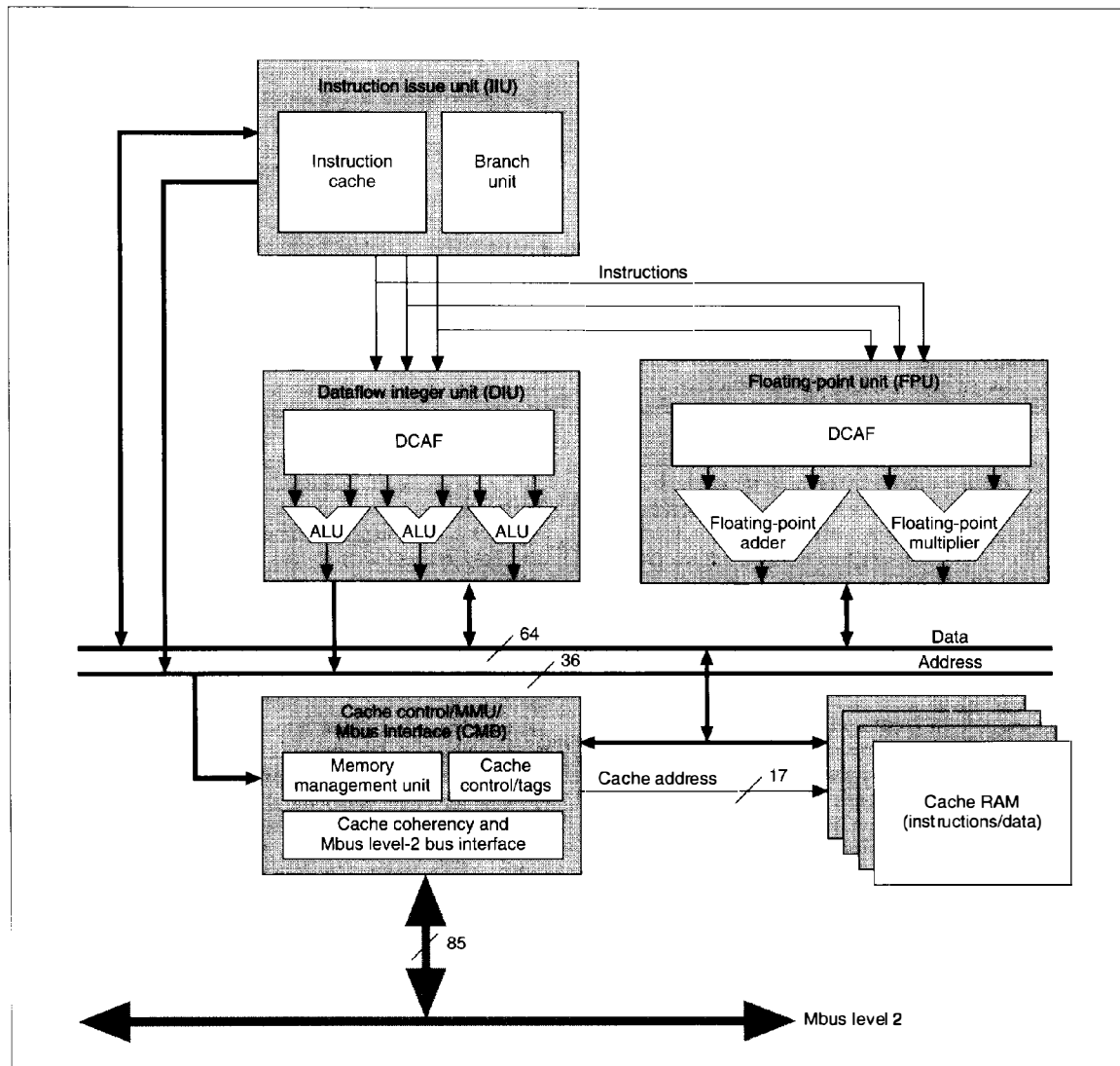


Figure 5. Lightning block diagram.

Update phases are duplicated in the dataflow integer unit and floating-point unit, and the Retire logic resides in the dataflow integer unit.

Instruction issue unit. This unit contains the self-aligning instruction cache (I-cache), the branch execution unit, portions of the DCAF relating to control flow, and the processor state registers.

The I-cache stores 32-bit Sparc instructions, using a four-

way, set-associative organization. Its self-aligning Fetch process is important in achieving maximum performance. Each line in the I-cache is eight instructions long. A branch to any of the last three of those eight instructions will be unable to fetch four instructions from the single line containing the branch target. The self-aligning I-cache avoids this problem. It does so by reading and concatenating two consecutive lines to guarantee supplying four instructions at all times.

The branch execution unit processes one branch instruction per clock cycle. It includes a specialized DRIS implementation called the branch shelf, which holds multiple speculative conditional branches. The IDs and results of up to three condition code-setting instructions (one from each of the two integer ALUs in the dataflow integer unit and the adder in the floating-point unit) appear every clock cycle. The branch shelf can update, schedule, and execute all shelved branches, and then pick the oldest incorrectly predicted one for a branch repair, in every cycle. The branch shelf also supplies the Retire process with the ID of the oldest speculative branch to prevent retiring that branch.

The instruction issue unit maintains the program counter. It computes four program counter results each clock cycle, one for each instruction issued, and shelves these values with the issued instructions. It also maintains miscellaneous processor state registers. For each of these that are changed infrequently by explicit instructions, the instruction issue unit maintains a one-entry shelf; issue is stalled only if a second instruction attempts to write the same state register before the first has retired.

The instruction issue unit fetches four instructions per clock cycle, allocates an ID to each, and sends up to three of them to both the dataflow integer unit and floating-point unit; the unit stalls when the DCAF is full. If one of the four instructions fetched on any clock cycle is a control-transfer instruction (branch, call, return), the unit executes that instruction itself.

Dataflow integer and floating-point units. The dataflow integer unit contains a DCAF, two integer ALUs, a memory address ALU, and the Retire logic. The floating-point unit contains a DCAF that holds a subset of the contents of the dataflow integer unit's DCAF, along with a floating-point adder and a floating-point multiplier. Both include copies of the ID allocation, dependency checking, and Schedule and Update logic.

The Issue logic in the dataflow integer unit allocates an ID to every instruction issued and enters the instruction into the DCAF; the floating-point unit allocates an ID to every instruction but enters only floating-point instructions into its smaller copy of the DCAF. The dataflow integer unit checks dependencies and invokes the Schedule and Update logic only on integer and memory-reference instructions; the floating-point instructions are in the dataflow integer unit's DCAF only for retirement purposes. The floating-point unit checks dependencies and invokes the Schedule and Update logic only on floating-point instructions. The Retire process is centralized in the dataflow integer unit, since it needs to consider the oldest shelved instructions, regardless of type. It can retire up to eight instructions per clock cycle: three that update integer registers or integer condition codes, two that update floating-point registers, one store instruction, and any two instructions of other types (control transfers, processor state updates).

The CMB. The cache controller/MMU/bus interface contains the cache tags and controller for the external cache, the MMU, and an Mbus interface. The instruction issue unit, dataflow integer unit, and floating-point unit communicate with the CMB over a 36-bit address bus and a 64-bit data bus. The instruction issue unit and dataflow integer unit each contain a translation look-aside buffer (TLB) to translate virtual addresses to physical addresses. The instruction issue unit translates each virtual instruction fetch address in parallel with searching its internal I-cache. If the instruction is not in the I-cache, the instruction issue unit sends the physical address to the CMB on the next clock cycle over the address bus; the instructions return on the data bus.

The dataflow integer unit provides physical addresses for all load and store instructions over the address bus. This unit receives integer data and sends integer store data over the data bus. Floating-point data transfers over the data bus to and from the floating-point unit. Finally, the dataflow integer unit uses the address bus to send update values for processor state registers (including the program counter for a computed branch) to the instruction issue unit.

***Lightning is a pioneering effort
to produce a superscalar,
out-of-order, speculative-
execution microprocessor.***

The cache controller looks up each physical address in its on-board cache tags. A hit results in sending an address to the external cache, with the data transferring to or from the instruction issue unit, dataflow integer unit, or floating-point unit directly over the data bus. A miss results in sending a request to the Mbus controller. The CMB continues to accept requests and respond to hits while processing outstanding misses.

The MMU portion of the CMB "walks" the page table in hardware. Either the dataflow integer unit or instruction issue unit, upon discovering that its on-board TLB does not contain a particular virtual address, sends that virtual address to the CMB. The CMB, assuming that the software has conformed to the Sparc reference MMU model, searches through up to three levels of page tables. If the CMB finds the addressed page-table entry, it returns that entry to the requesting TL; otherwise it returns an exception indication, which causes an interrupt if enabled.

Design challenges

Lightning is a pioneering effort to produce a superscalar, out-of-order, speculative-execution microprocessor. It runs counter to the conventional wisdom that applying these techniques to the design of a microprocessor will lead to significant increases in three areas that will effectively negate all advantages of these design techniques. These areas are:

- design complexity (as measured by development time and money and by uncertainty in design verification),
- clock cycle period, and
- die size.

We address these concerns from a unique perspective. We have completed the logic design of the Lightning processor, verifying (with gate-level simulation) its compliance with the *Sparc Architecture Manual*¹⁰ and its estimated worst-case commercial clock speed of 40 MHz (based on static and dynamic timing analysis).

Design complexity. Two perceptions are likely to fuel the anxiety regarding the complexity of out-of-order execution. They are 1) the belief that a vast number of machine states need to be managed as part of the logic design process, and 2) the uncertainty that all possible states can be identified for design verification purposes. Perhaps these concerns would be real if the design approach were to derive all possible machine states and their relationships directly from a high-level architecture description and then cast them into low-level logic. Clearly the design of a processor as complex as Lightning could not be approached in this way.

The alternative approach is to develop a rigorous abstract model that covers all aspects of the processor's operation. We developed such a model over a period of several years. The model defines methods of ensuring correct out-of-order speculative execution with precise traps and interrupts, independent of details of a particular instruction-set architecture. We refined and validated this model with extensive simulation using a register transfer-level model.

Applying the Metaflow architectural model to the Sparc instruction-set architecture yielded a correct logic design in a period of months rather than years. An important factor in achieving this high design productivity was the use of a hardware description language coupled with logic synthesis tools.

The scalable Metaflow architecture lets the designer choose the level of design complexity, clock speed, size, and performance best suited to a given application. For example, by adjusting the size of the DRIS, the designer can control the degree of out-of-order execution, and hence the performance target.

It will come as no surprise to designers of modern microprocessors that the most difficult verification problems arose

in the interaction between caches (first and second level), the MMU, and the Mbus. These problems include enforcing a coherent, totally, or partially ordered memory model in the context of out-of-order loads; maintaining correct paging information in the MMU; and dealing with error-reporting mechanisms related to the Mbus. These issues presented more formidable design and verification tasks than basic out-of-order execution issues.

Clock period. The clock speed of a microprocessor results from many trade-offs. Designers typically trade off data path speed and control logic speed against the number of pipeline stages. Data path delays in a given superscalar processor design will not be greatly affected by adding out-of-order execution features. Control logic delays, however, will be significantly different when out-of-order execution is added to the design.

The very essence of out-of-order execution, the decoupling of scheduling from issuing, necessarily adds a pipeline stage (compare Figure 2 and Figure 1). The bypass from the execution units to the Schedule phase, however, means that, just as in Figure 1, the result produced by an arithmetic instruction in one clock cycle can be used as an operand in another arithmetic instruction on the next cycle (one-clock-cycle result-to-operand bypass).

**Designers can choose
the level of design
complexity, clock speed,
size, and performance
best suited to a
given application.**

The added pipeline stage for scheduling does, however, increase the delay between fetching a condition code-setting instruction and returning the condition codes to control fetching after a subsequent conditional branch. This increased delay has a large adverse effect on performance in an in-order superscalar design, due to attendant stalls in the Fetch stage (see Figure 4). But with out-of-order speculative execution, the Fetch unit never stalls for condition codes. The only time delay in returning condition codes affects performance is when a branch has been incorrectly predicted, which is relatively rare. In these cases, the total of the delays due to decoupled scheduling and branch repair is large

enough that adding one more clock cycle has little impact on overall performance.

The dataflow instruction scheduler logic in the Metaflow architecture may lengthen the basic clock cycle when compared to a conventional processor, because this logic cannot be pipelined if it is to achieve a one-clock, result-to-operand bypass. Within each clock cycle the scheduler must examine the state of a large number of instructions stored in the DRIS, determine which ones are ready, and select a prioritized subset for execution, subject to execution resource constraints. Fortunately, we can control this critical path through the scheduler by exploiting locality in the instruction readiness patterns.

Die size. LSI Logic is implementing the Lightning processor in CMOS using a combination of custom cells and standard ASIC technology. It implemented speed-critical and area-intensive logic as custom cells, and converted all others into sea-of-gates logic using automated synthesis tools. Scan paths provide access to all logic elements for the purposes of testing (and design validation). As of this writing, the final layout of the Lightning chips has not been completed, so we do not have exact die size figures; preliminary estimates indicate that the four-chip set should be commercially viable.

Any superscalar design, with its multiple parallel functional units, will consume a vast amount of silicon for the many buses conveying operands and results to and from the many functional units. Much of the additional logic required to implement the Metaflow out-of-order speculative execution fits within this massive "highway system."

Performance estimates

Although prototypes of LSI Logic's Lightning Sparc processor are not yet available, we have estimated performance. We based these estimates on a C-language, register transfer-level description of the Lightning processor. This simulation accurately models all elements of the Sparc processor chip set, including the pipelined execution units, internal buses, caches, the MMU, and the Mbus. With this model, we studied numerous benchmark programs to analyze trade-offs of the Lightning design.

The Lightning simulation program is not trace-driven, thereby sacrificing speed for accuracy. The simulator executes Sparc programs (in the form produced by standard Sparc compilers) at a rate of about 1,000-2,000 instructions per second (on a Sparc Server 490). The simulator program also contains a facility that executes "snapshots" of the state of a running Sparc workstation, allowing simulated executions that include Unix kernel code.

Because of the limited speed of the simulator program, we could not run the SPEC benchmark suite¹¹ in its entirety. However, we can estimate a performance by intelligent sampling of an appropriate number of sections of the

Table 5. Estimated Lightning performance.

Benchmark	Rate
Dhrystone (Version 1.1)	116,000/second (92 VAX MIPS)
Linpack (100 × 100)	18 Mflops
Specmark	> 80 (VAX MIPS)

benchmark program suite. We ran benchmark programs such as Dhrystone and Linpack to completion.

Table 5 lists projected results for these three benchmarks. The Linpack results represent double-precision floating-point numbers. On the Specmark program suite, the Lightning processor shows a range of 1.4 to 3.0 Sparc instructions per clock cycle.

This simulation study of the Lightning implementation of the Metaflow architecture produced the following observations:

- Lightning sustains rates of more than two Sparc instructions executed per clock cycle on many programs. The capability of issuing and executing instructions out of order and speculatively is the key reason for this performance.
- The processor tends to constantly execute in speculation (at least one predicted conditional branch instruction remains outstanding at all times.)
- The processor is somewhat insensitive to the quality of the compiler's code generator, especially with respect to code scheduling and register allocation.
- Performance is relatively insensitive to memory latency for data in programs having sufficient instruction-level parallelism.
- The limits on performance of the Lightning implementation running the SPEC benchmark suite include memory bandwidth, floating-point execution unit bandwidth, branch prediction accuracy, calculated branch address latency, and instruction cache hit rate.

Analysis of the code produced by the Sparc compilers presently available from Sun Microsystems shows a number of performance-limiting effects that could be mitigated by relatively straightforward changes to the code-generation algorithms, especially with respect to the scheduling and use of conditional branch instructions.

THE METAFLOW ARCHITECTURE EXPLOITS inherent instruction-level parallelism in conventional sequential programs without relying on optimizing compilers. The hardware shelves instructions that would cause stalling due to data or control dependencies and proceeds immediately to issue succeeding independent instructions. Conditional branches that would stall execute speculatively in a predicted direction, and multiple speculative conditional branches can be active simultaneously. Discovering that a speculative branch was incorrectly predicted invokes branch repair. Instructions are scheduled for execution only when all their operand values have been computed. The results of an instruction's execution are shelved with the instruction and then retired to permanent storage in program issue order. Shelving results accomplish automatic register renaming and a coherent processor state on interrupt.

The Lightning project has established that superscalar, out-of-order, speculative-execution microprocessors are indeed practical today, given the current state of silicon technology and high-level logic design tools. The increasing number of ongoing (and planned) commercial development efforts that are described in the same terms reinforce this conclusion.

The discussions and examples in this article relate to superscalar implementations of RISC machines, but the fundamental problem of stalling arises in other (CISC) processor designs as well. Designers can apply the same techniques for avoiding stalls and maximizing the use of execution units to other instruction-set architectures and improve performance significantly. Specific candidates are the older CISC architectures, which because of fundamental instruction-set limitations will require out-of-order speculative execution for future substantial increases in processor performance. □

Acknowledgments

LSI Logic Corporation and Hyundai Electronics Industries Co., Ltd. jointly supported development of the Lightning implementation of the Metaflow out-of-order, speculative-execution architecture.

References

1. N.P. Jouppi and D.W. Wall, "Available Instruction-Level Parallelism for Superscalar and Superpipelined Machines," *Proc. Third Int'l. Conf. Architectural Support for Programming Languages and Operating Systems*, IEEE Computer Society Press, Los Alamitos, Calif., Apr. 1989, pp. 272-282.
2. J. Hennessy and T. Gross, "Postpass Code Optimization of Pipeline Constraints," *ACM Trans. Programming Languages and*

Systems, Vol. 5, No. 3, New York, July 1983, pp. 422-448.

3. R.M. Tomasulo, "An Efficient Algorithm for Exploiting Multiple Arithmetic Units," *IBM J. Research and Development*, Vol. 11, No. 1, Jan. 1967, pp. 25-33.
4. R.M. Keller, "Look-Ahead Processors," *ACM Computing Surveys*, Vol. 7, No. 4, Dec. 1975, pp. 177-195.
5. J.E. Smith and A.R. Pleszkun, "Implementing Precise Interrupts in Pipelined Processors," *IEEE Trans. Computers*, Vol. 37, No. 5, May 1988, pp. 562-573.
6. G.S. Sohi, "Instruction Issue Logic for High-Performance, Interruptible, Multiple Functional Unit, Pipelined Computers," *IEEE Trans. Computers*, Vol. 39, No. 3, Mar. 1990, pp. 349-359.
7. J.C. Huck and M.J. Flynn, *Analyzing Computer Architectures*, IEEE CS Press, 1989.
8. A.J. Smith and J.K.F. Lee, "Branch Prediction Strategies and Branch Target Buffer Design," *Computer*, Vol. 17, No. 1, Jan. 1984, pp. 6-22.
9. W.M. Hwu and Y.N. Patt, "Checkpoint Repair for High-Performance Out-of-Order Execution Machines," *IEEE Trans. Computers*, Vol. 36, No. 12, Dec. 1987, pp. 1496-1514.
10. *The Sparc Architecture Manual*, Version 8, Part 800-1399-12, Sun Microsystems, Inc., Mountain View, Calif., Dec. 1990.
11. *SPEC Release 1.0*, Standards Performance Evaluation Corporation, c/o Waterside Associates, Fremont, Calif., Sept. 1989.



Val Popescu, the founder and president of Metaflow Technologies and the original contributor to the Metaflow architecture, has more than 20 years of experience in computer architecture and processor design. His professional interests include processor architecture and multiprocessing.

Popescu received his engineering degree from the Polytechnical Institute of Bucharest, Romania.



Merle Schultz is one of the architects and early developers of the Metaflow architecture and a cofounder of Metaflow Technologies. He held responsibility for the simulator used to develop and tune the architecture, as well as the implementation of the CMB part of Lightning. His professional interests include high-performance processor design and architecture.

Schultz received a BA degree in physics from Cornell University.



John Spracklen is a contributing architect of the Metaflow architecture and also cofounded Metaflow Technologies. He is manager of both the dataflow integer unit and floating-point unit designs for Lightning. He became involved with the Metaflow architecture in October 1987, after working on local area network projects for Burroughs (now Unisys) Corporation. His research interests include superscalar and multiprocessing architectures.

Spracklen received a BS degree from California Polytechnic University, San Luis Obispo, in electronics engineering technologies.



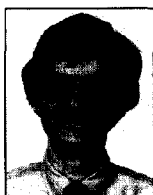
Bruce Lightner is the vice president of development at Metaflow Technologies and a coinventor of the Metaflow architecture. He has 15 years of experience in computer systems engineering, ranging from low-level hardware design to high-level systems software. His current research interests include computer architecture and high-performance processor design.

Lightner received a BA degree in chemistry from the University of California at San Diego, followed by several years of postgraduate study in geo/cosmochemistry. He is a member of the IEEE.



Gary Gibson, another cofounder of Metaflow Technologies and an architect of the Metaflow architecture, is a senior scientist managing the CMB design. His research interests include computer architecture, high-performance CPU design, multiprocessor systems, large-scale processor-memory interconnection networks, and compiler technology.

Gibson received a BS degree from the University of California at San Diego in electrical engineering.



David Isaman is a senior staff scientist with Metaflow Technologies responsible for the architecture and design of the instruction issue unit of the Lightning microprocessor. He has worked with the architecture of high-speed processors and local-area networks and served on the Computer Science faculty at the University of California at San Diego.

Isaman received a BS degree from the California Institute of Technology in Pasadena and an MS and PhD in computer science from the Massachusetts Institute of Technology.

Reader Interest Survey

Indicate your interest in this article by circling the appropriate number on the Reader Service Card.

Low 153

Medium 154

High 155

Readers may direct questions concerning this article to the authors at Metaflow Technologies, Inc., 6725 Mesa Ridge Road, Suite 100, San Diego, CA 92121.

June 1991 73